* **Always Remember:→** Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. —VHiren

* Problem:-

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were Broken into on the same night.

Given an integer array houses representing the amount of money of each house, Return the maximum amount of money you can rob tonight without alerting the police.

Input: houses = [1, 2, 3, 1] , $n = 4$
Output: 4

→ Can't rob adjacent houses

→ if rob at i^{th} house then skip $i+1^{th}$

(index) houses →	0	1	2	3
money →	1	2	3	1

1 ⇝ rob 0^{th} house, then skip 1^{th} house and rob 2^{th} house.

Total money robbed $= 1+3 \Rightarrow 4$

2 ⇝ rob 1^{th} house, then skip 2^{th} house and rob 3^{th} house.

Total money robbed $= 2+1 \Rightarrow 3$

⇒ From this maximum ways, answer is $\max(4, 3) \Rightarrow \underline{4}$

→ Here we could see at each house or index,
we have two options
- → rob it and skip next one
- → don't rob and move to next one

* How Question Provides Hints to think Recursive?

(i) Tries to give you options to explore

(ii) Tells you can't go greedy, must explore all possibilities/options to build result

→ So at any Index or House the options we have :-

(index) houses →	0	1	2	3
money →	1	2	3	1

Taking money of i^{th} house

robHouse = houses[i] + get maximum money from ($i+2^{th}$ index)

skipHouse = get maximum money from ($i+1^{th}$ index)

→ Find recursively

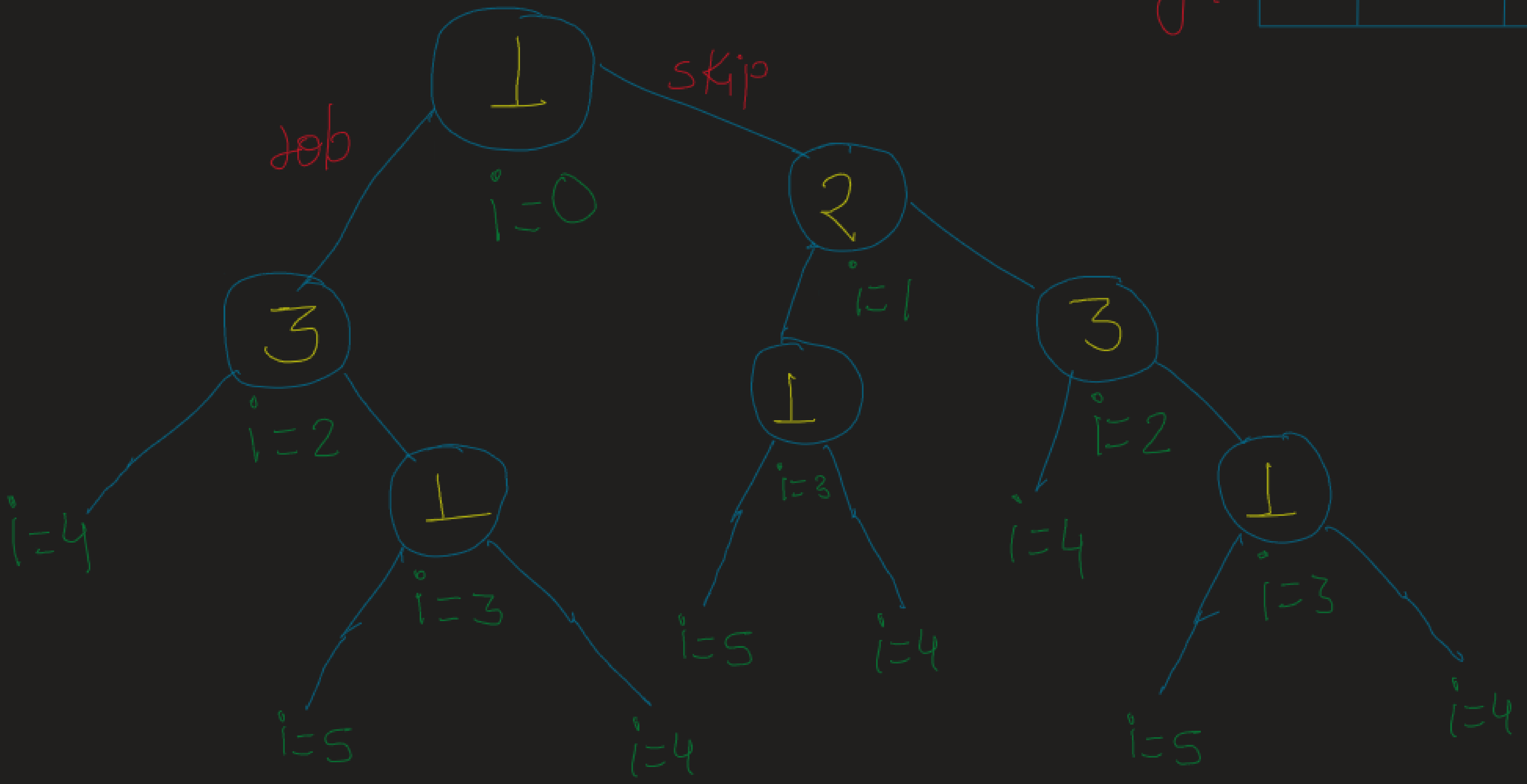
→ overall we need maximum money can rob, so take maximum from both; max(robHouse, skipHouse) and return it

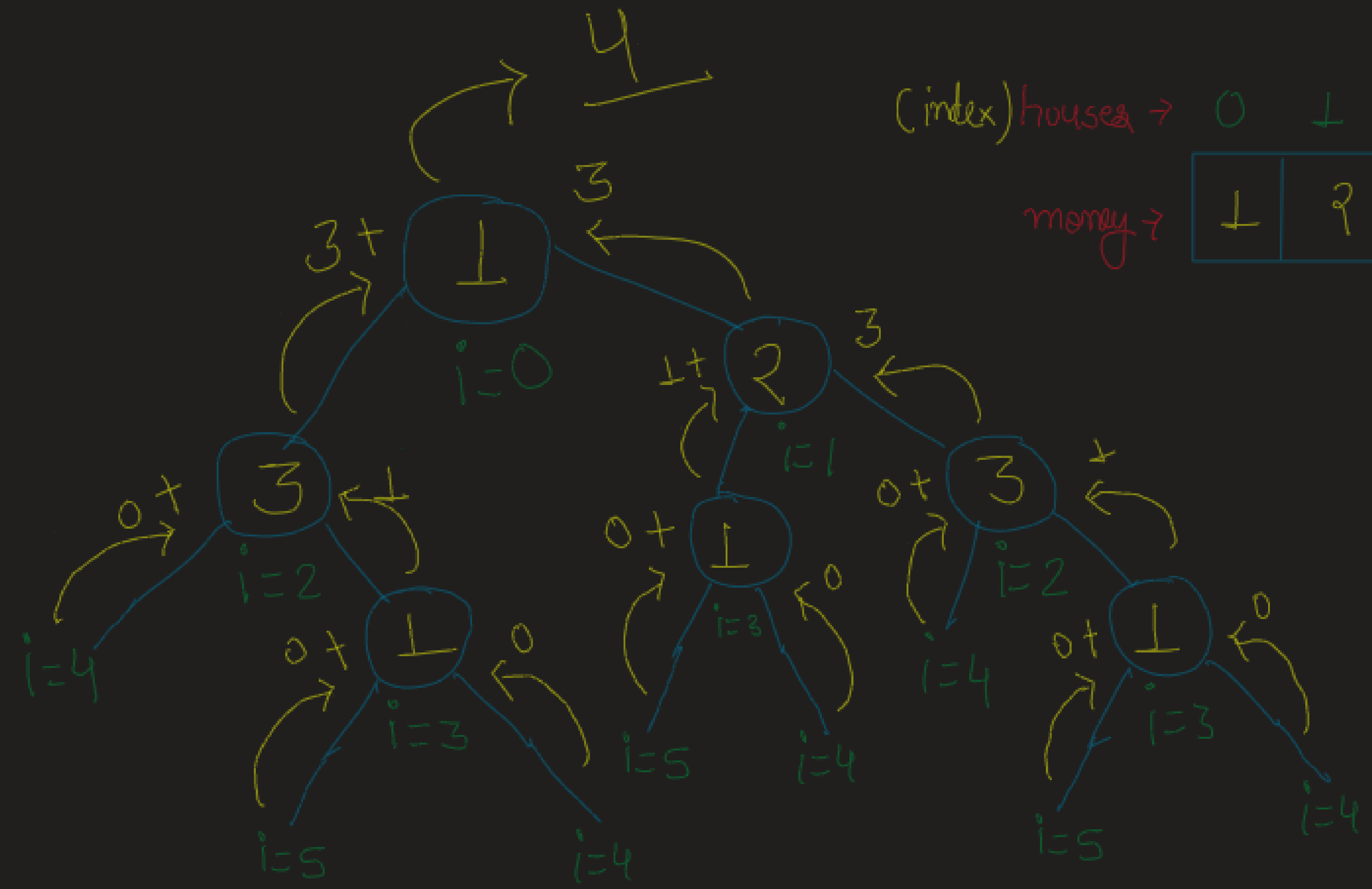
→ overall we need maximum money can rob, so take maximum from both; max(robHouse, skipHouse) and return it

→ In recursion we will be going to each index and will apply these 2 options to each of them, because we can't go greedy, we must explore all ways to build maximum result (Best result)

→ so the point when no index exists, is when ($i >= n$). So this is our stop condition.
so logically no house exists when ($i >= n$)
hence could rob no money, 0 money.

(index) houses →	0	1	2	3
money →	1	2	3	1





(index) houses →	0	1	2	3
money →	1	2	3	1

robHouse = houses[i] + get maximum money from ($i+2^{th}$ index)

skipHouse = get maximum money from ($i+1^{th}$ index)

Base case → if (i >= n) return 0

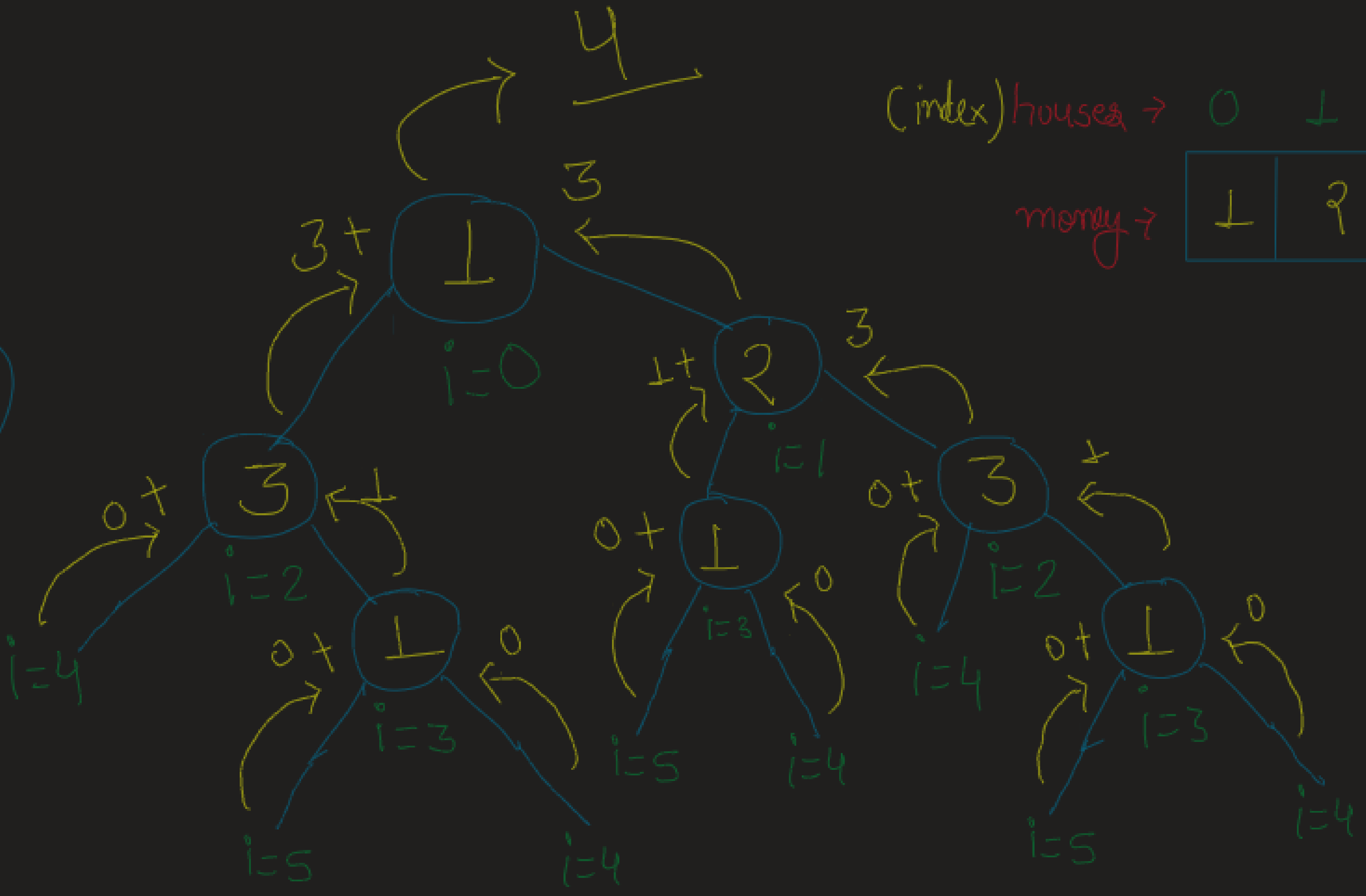
→ Base case → if (i >= n) return 0

* Pseudo code:-

```
int rob(i, houses) {

    robHouse = houses[i] + rob(i+2, houses)
    skipHouse = rob(i+1, houses)

    return max(robHouse, skipHouse)
}
```

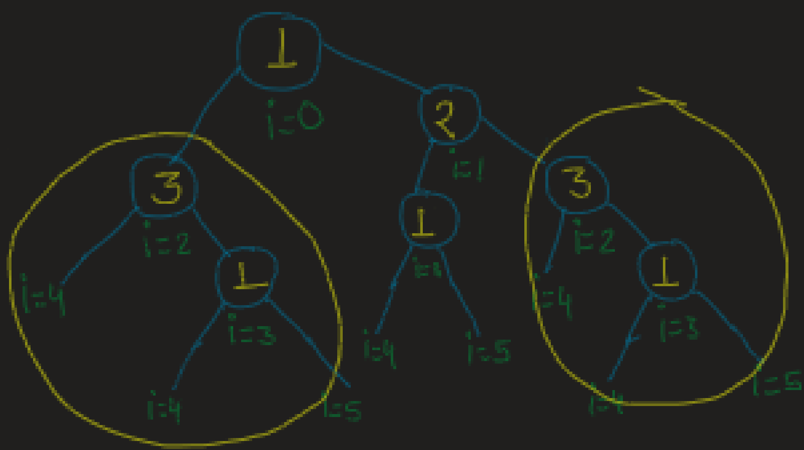


(index) houses →	0	1	2	3
money →	1	2	3	1

* Time → $O(2^N)$

→ for any index we're exploring two options / possibilities.

: Why It's taking Exponential time ?



→ Overlapping Sub problems (Repitative work)

→ Means when n is increases, the amount of sub-problem increases

→ solving this repetative problems again and again is increasing time.

→ This concludes that we could optime time in our recursion.

→ Now this hints to DP.

→ so for recursion, we apply Top-Down DP, so lets do Memoization.

→ Now memoize it (Apply DP)

→ let's code it